

Practical No. 10

Name : Sharad Bobade

Class : SY- B

Branch – AI & DS

Roll no : B-41

Aim - Version Control using Git & GitHub Questions :

1. What is the difference between Git and GitHub? Ans

: Git vs GitHub

- Git is a Version Control System (VCS)
- Created by Linus Torvalds in 2005
- Works completely offline on your local machine
- Tracks every change made to your code
- Stores full project history locally
- Free and open-source
- Used via command line (terminal)
- Alternatives: Mercurial, SVN
- GitHub: GitHub is a cloud-based hosting platform for Git repos
- Founded in 2008, bought by Microsoft in 2018
- Requires internet connection
- Adds collaboration features: Pull Requests, Issues, Actions
- Stores your code remotely on servers
- Has free and paid plans
- Alternatives: GitLab, Bitbucket

2. Explain the difference between:

- **git add**

- git commit

Ans : git add:

- Moves files from Working Directory → Staging Area
- Tells Git which changes to include in next commit
- Does NOT save permanently
- Can stage one file, multiple files, or all files
- Reversible using git restore --staged

Commands:

Git add file.txt → one file

Git add . → all files

Git add *.js → all JS files

Git add -p → select parts interactively

- Git commit:
- Moves files from Staging Area → Local Repository
 - Creates a permanent snapshot in history
 - Each commit gets a unique SHA-1 hash ID
 - Requires a commit message • Records author, timestamp, and changes

Commands:

Git commit -m "message" → basic commit

Git commit --amend → edit last commit

Git commit -a "message" → add + commit together

3.What is the staging area in Git?

Ans : • Also called the Index

- It is a middle layer between working directory and repository
- Physically stored in .git/index file
- Lets you selectively choose what to commit
- You can modify 10 files but commit only 2

- Helps create clean, logical commits
- You can review staged changes before saving

- Commands:

Git status → see staged/unstaged files Git

diff –staged → see staged changes Git

restore –staged file → unstage a file -

4 States of a file:

- Untracked → new file Git doesn't know about
- Modified → tracked file with new changes
- Staged → ready to be committed
- Committed → permanently saved in history

4 What happens if you modify a file after committing it?

Ans : • The old committed version stays safe in history

- Modified file appears as “modified” in working directory
- Git detects change by comparing file hash with last commit
- Git status shows it under “Changes not staged for commit”
- You must add and commit again to save new version
- Each commit gets a new unique hash
- Old commit is still accessible anytime
- You can go back using git checkout <old-hash>
- Nothing in Git is truly lost after committing
- To fix last commit quickly: git commit –amend Process after modifying:

Modify file → git add → git commit → new snapshot created

5.Difference between:

- git push

- git pull

Ans : git push:

- Uploads local commits → remote repository (GitHub)
 - Shares your work with teammates
 - Only pushes committed changes (not just staged)
 - May be rejected if remote has newer commits
- Commands:
- Git push origin main → push to main branch
 - Git push origin branch-name → push specific branch
 - Git push -u origin main → set default upstream
 - Git push -force → force overwrite (dangerous)

pull:

- Downloads remote changes → local machine
 - Keeps your local repo up to date
 - Combines: git fetch + git merge
 - Can cause merge conflicts if same lines were edited
- Commands:
- Git pull origin main → pull from main
 - Git pull -rebase → pull and rebase instead of merge

6 .What is Version Control? Differentiate between Centralized v/s Distributed VCS.

Ans : Version Control:

- System that records changes to files over time
 - Tracks who, what, when, and why something changed
 - Allows reverting to any previous version
 - Enables parallel work without overwriting each other • Essential for team software development
- Centralized VCS (CVCS):
- One central server holds the entire repository
 - Developers check out only one version (not full history)
 - All operations need server/internet connection

• Examples: SVN, CVS, Perforce - Advantages:

- Simple to understand
 - Admin has full control
 - Good for binary/design files
- Distributed VCS (DVCS):
- Every developer has full copy of entire repo including history
 - Most operations happen locally (fast) • Remote server used only for syncing/sharing

• Examples: Git, Mercurial - Advantages:

- Fully offline capability
- Every clone = complete backup
- Very fast operations
- Easy branching and merging • No single point of failure **Theory :**

```

MINGW64:/c/Users/JS COMPUTERS/sds
JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~
$ git config --global user.name "Sharad Bobade"

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~
$ git config --global user.email "bobadesharad2323@gmail.com"

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~
$ mkdir sds

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~
$ cd sds

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds
$ git init
Initialized empty Git repository in C:/Users/JS COMPUTERS/sds/.git/

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$ touch index.html

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$ git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with "git config set advice.addEmptyPaths false"

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$ git commit -m "First commit"
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$ git add .

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$ git commit -m "First commit"
[master (root-commit) 8bc3735] First commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 index.html

JS COMPUTERS@DESKTOP-QTJHVDD MINGW64 ~/sds (master)
$

```

Git - Install for Windows | bobadesharad2323-sudo/sds | Ask Gemini

github.com/bobadesharad2323-sudo/sds

bobadesharad2323-sudo / sds

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

sds Public

Pin Watch 0 Fork 0 Star 0

Set up GitHub Copilot

Use GitHub's AI pair programmer to autocomplete suggestions as you code.

Get started with GitHub Copilot

Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

